14-APRIL-2026

API SECURITY RISK ANALYSIS REPORT

API TESTED:https://jsonplaceholder.typicode.com

LAKSHMI PAVAN AKULA

TASK-03

ABOUT THE PROJECT

- PROJECT TYPE: API Security Analysis
- DOMAIN: Cyber Security / Application Security
- FOCUS AREA: API Vulnerability Assessment

OBJECTIVE

- Analyze a public API for security risks
- Identify vulnerabilities in authentication and authorization
- Detect excessive data exposure
- Evaluate API response behavior and headers
- Classify risks based on severity (Low / Medium / High)
- Explain security issues in simple language

SCOPE

- Analysis is performed on a public demo API (JSONPlaceholder)
- Only read-only requests (GET) were used
- No exploitation, attack, or system damage was performed
- The analysis is conducted strictly for educational purposes
- Focus is on risk identification and prevention, not hacking

TOOLS USED

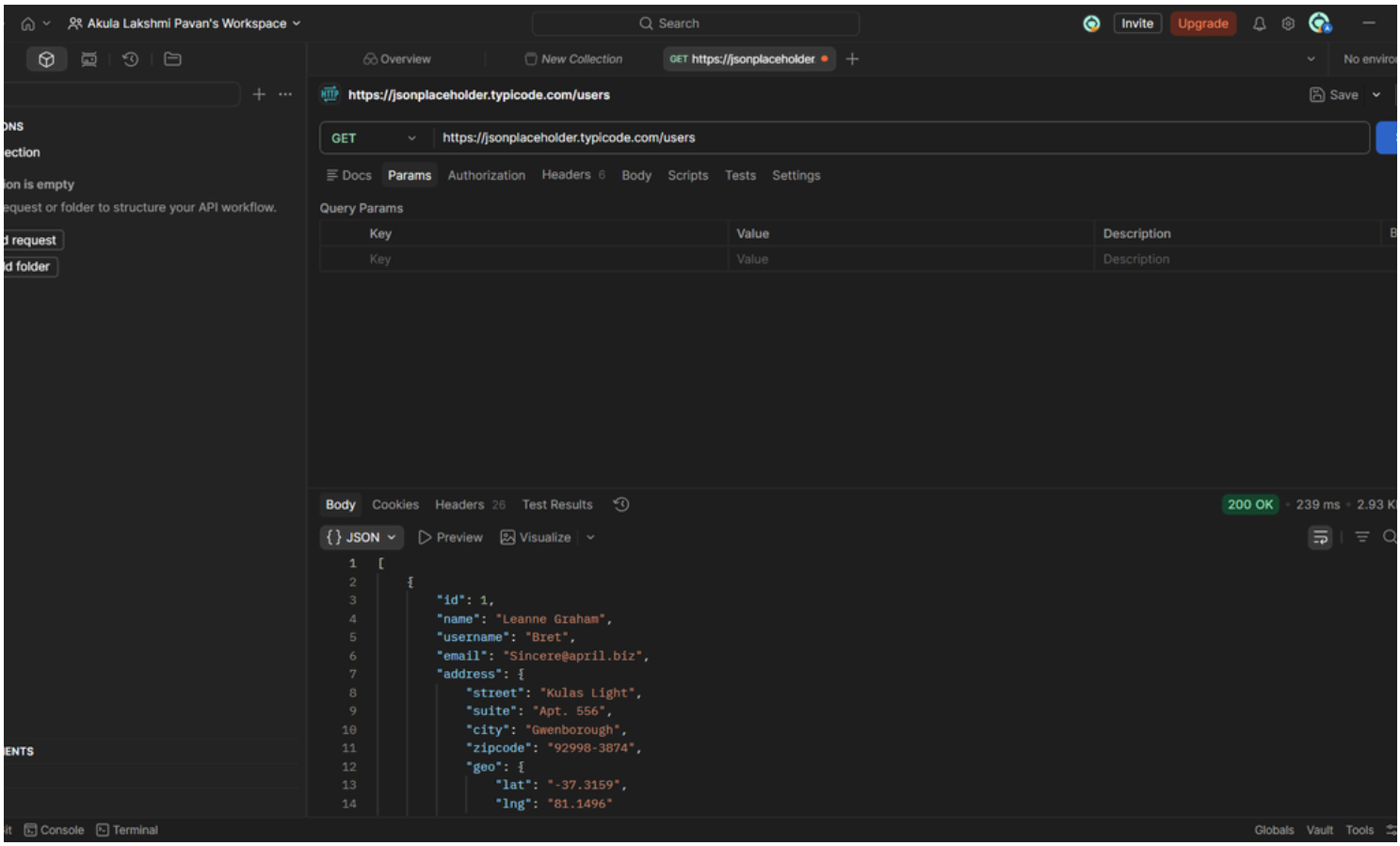
- Postman – API request testing and response analysis
- Kali Linux – Security-focused testing environment
- Browser Developer Tools – Inspect headers and requests
- Manual Analysis Techniques – Logical security evaluation

API TESTING & OBSERVATIONS

Endpoint Tested:/users

Observations:

- API returns user details including name, email, address, and phone
- No authentication required
- Public access is allowed
- No request limitations



ADDITIONAL ENDPOINTS TESTED: /USERS/1 ,/USERS/2

Observations:

- Any user data can be accessed directly
- No identity verification
- Same data structure for all users

FINDINGS & RISK ANALYSIS

1. No Authentication Mechanism

Issue:

The API allows access without requiring login credentials or tokens.

Severity: High

Explanation:

Any user or attacker can access the API without verification.

There is no system to confirm whether the request is from an authorized user.

Why it matters:

This can lead to:

- Unauthorized data access
- Data leakage

Remediation:

- Implement JWT (JSON Web Token) authentication
- Use API keys or OAuth
- Restrict access to authorized users only

2. EXCESSIVE DATA EXPOSURE

Issue:
The API returns full user details including unnecessary fields.
Severity: Medium
Explanation:
The API exposes more data than required, such as:

- Email
- Phone number
- Address

Why it matters:
Sensitive data exposure increases the risk of misuse and privacy breaches.
Remediation:

- Return only required fields
- Use response filtering
- Avoid exposing sensitive information

3. BROKEN AUTHORIZATION

Issue:
Users can access data of other users without restriction.
Severity: High
Explanation:
The API does not verify whether a user is allowed to access specific data.
Why it matters:
This can lead to:

- Unauthorized access
- Data breaches
- Identity misuse

Remediation:

- Implement Role-Based Access Control (RBAC)
- Validate user identity before providing data
- Restrict access based on roles

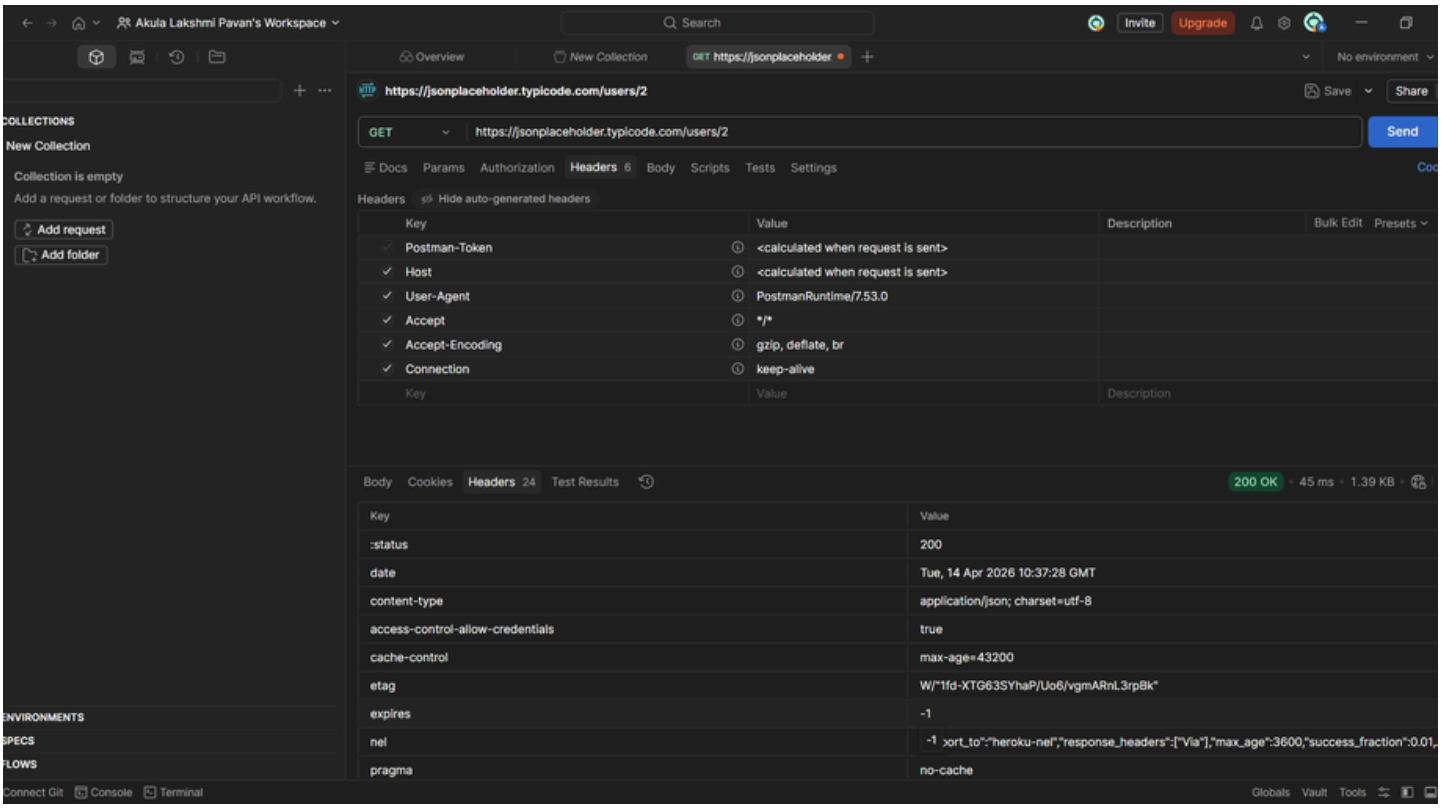
4. NO RATE LIMITING

Issue:
The API allows unlimited requests.
Severity: Medium
Explanation:
There is no limit on how many requests can be sent to the API.
Why it matters:
This can lead to:

- API abuse
- Server overload
- Denial of Service (DoS) attacks

Remediation:

- Limit requests per user/IP
- Use API gateways
- Implement throttling



EVIDENCE IMAGE

HOW API SECURITY RISKS WORK

- Attacker sends request to API
- API responds without authentication
- Sensitive data is exposed
- Attacker collects and misuses data
- System security is compromised

COMMON API SECURITY ISSUES

- Missing authentication
- Weak authorization
- Excessive data exposure
- Lack of rate limiting
- Poor input validation

DO’S AND DON’TS

- DO:
- Use secure authentication methods
- Validate user access
- Limit API responses
- Monitor API usage
- DON’T:
- Expose sensitive data
- Allow unrestricted access
- Ignore security best practices
- Skip validation checks

PREVENTION GUIDELINES

- Always implement authentication mechanisms
- Restrict API access using authorization controls
- Apply rate limiting to prevent abuse
- Validate all inputs
- Monitor API traffic

CONCLUSION

The analysis shows that the tested API lacks essential security controls such as authentication, authorization, and rate limiting.
Although the API is a demo system, By implementing proper security measures, organizations can protect their APIs from unauthorized access and data breaches.